

SIMATS ENGINEERING

ASSESSMENT TOOL – 3

COURSE: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA1703

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.

ANNEXURE A

REVERSE ENGINEERING

Assessment Tool Weightage: 20% | Duration: 1 Hour | Total Marks: 50

CODE OF CONDUCT / DECLARATION

I Hashini S / 192524035 (Name / Reg No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Hashini S

1. INTRODUCTION TO REVERSE ENGINEERING IN AI SYSTEMS

Reverse engineering is the systematic process of studying the observable behaviour of an existing system and reconstructing a plausible internal architecture, data flow, processing logic and decision mechanism. In artificial intelligence, the objective is not necessarily to reproduce proprietary source code. Instead, the analyst infers how inputs are collected, transformed, distributed, scored and converted into actions.

Modern AI applications increasingly depend on distributed computing because the volume, velocity and variety of data are too large for a single machine. PySpark provides a practical framework for distributed data processing. Its Resilient Distributed Dataset (RDD) abstraction divides data into partitions and allows transformations and actions to be executed across a cluster.

Intelligent agents add another layer of autonomy. An agent can observe an environment, maintain a state or knowledge base, evaluate objectives and take actions. In the systems considered in this assignment, agents may recommend products, adjust traffic signals, issue fraud alerts, adapt learning content, or initiate cloud scaling.

1.1 Objectives of the Reverse Engineering Exercise

- Identify the likely sources and forms of operational data.
- Reconstruct the major stages of a distributed AI pipeline.
- Explain where PySpark and RDD operations can be applied.
- Distinguish RDD transformations from actions.
- Identify intelligent agents and their responsibilities.
- Explain how real-time or near-real-time decisions can be produced.
- Discuss fault tolerance, scalability, security and ethical issues.

1.2 General Reverse Engineering Methodology

1. Observe system inputs, outputs, response time and visible behaviour.
2. Identify likely data sources and event streams.
3. Map raw data to preprocessing and feature-engineering stages.
4. Determine where distributed processing is necessary.
5. Represent likely transformations using RDD operations.
6. Identify the model or rule engine responsible for scoring.
7. Map scores to agent decisions and system actions.
8. Evaluate feedback loops and continuous updating.
9. Assess reliability, latency, security and scalability.

2. PYSPARK AND RDD FUNDAMENTALS

Apache Spark is a distributed computing framework designed for large-scale data processing. PySpark is its Python API. An RDD is a distributed collection of records that can be processed in parallel across cluster nodes. RDDs are immutable, partitioned and designed to support fault-tolerant computation.

2.1 RDD Transformations

Operation	Purpose	Typical AI Use
map()	Transforms each record	Feature creation
filter()	Keeps records satisfying a condition	Remove invalid events
flatMap()	Produces zero or more outputs per record	Token/event expansion
reduceByKey()	Aggregates values by key	Counts, sums, statistics
groupByKey()	Groups values by key	Grouped event analysis
distinct()	Removes duplicate values	Unique users/items
sortBy()	Sorts records	Ranking or reporting
join()	Combines datasets by key	User + product features

2.2 RDD Actions

Action	Purpose
collect()	Returns records to the driver; suitable only for small results
count()	Counts records
first()	Returns first record
take(n)	Returns a limited number of records
reduce()	Combines records using a function
saveAsTextFile()	Writes distributed output

2.3 Example RDD Pipeline

A generic AI pipeline can be represented as:

Raw Events → RDD Creation → Cleaning → Feature Extraction → Aggregation → Model/Rules → Agent Decision → Action → Feedback

For example, a transaction stream can be converted into an RDD, filtered for valid records, mapped into numerical features, grouped by account, scored for risk, and passed to a fraud-alert agent.

2.4 Fault Tolerance

RDDs maintain lineage information describing how a dataset was derived. If a partition is lost, Spark can recompute the missing partition from the lineage rather than requiring the entire computation to restart. This property is particularly useful in large distributed AI pipelines.

3. INTELLIGENT AGENT ARCHITECTURE

An intelligent agent perceives information from its environment and selects actions according to objectives, rules, learned models or a combination of these. A distributed AI application may contain multiple specialized agents instead of a single central decision-maker.

Agent Component	Responsibility
Perception	Receives events and sensor/data inputs
State / Memory	Maintains relevant historical information
Reasoning	Applies rules, models or policies
Decision	Selects an action based on current evidence
Actuator	Executes the selected action
Feedback	Observes results and updates future decisions

3.1 Agent Interaction Pattern

A useful architecture for the five systems is:

Environment → Sensors / APIs / Logs → Distributed Data Layer → AI Analytics → Agent → Action → Environment

Multiple agents may cooperate. A monitoring agent may detect an abnormal condition, a decision agent may estimate severity, and a response agent may execute a predefined action. Coordination reduces the workload of a single controller and allows specialized policies.

3.2 Time-Constrained Decisions

For systems such as traffic control and fraud detection, the agent must operate under strict latency constraints. This means expensive operations should be minimized on the critical path. Precomputed statistics, cached models, partition-aware processing and lightweight rules can reduce response time.

4. QUESTION 1 – REVERSE ENGINEERING A RECOMMENDATION SYSTEM

Scenario: A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. The objective is to infer a plausible architecture using PySpark, RDD operations and intelligent agents.

4.1 Observed Inputs and Data Sources

- Product catalogue: product ID, category, brand, price and attributes.
- Customer profile: account-level preferences and historical interactions.
- Behavioural events: views, searches, clicks, cart additions and purchases.
- Context: time, device, location region and session information.
- Feedback: ratings, returns, skips and recommendation clicks.

4.2 Reconstructed Data Pipeline

The likely pipeline is:

User Events → Event Collection → Distributed Storage/Stream → PySpark Processing → Feature RDDs → Candidate Generation → Ranking → Recommendation Agent → Display → Feedback

4.3 RDD-Based Processing

Events can be represented as tuples such as (user_id, product_id, event_type, timestamp). A filter operation can remove invalid events. A map operation can convert events into features. reduceByKey() can aggregate interaction counts per user-product pair. join() can combine behavioural features with product metadata.

Illustrative pseudocode:

```
events = raw.map(parse_event)

valid = events.filter(is_valid)

pairs = valid.map(lambda e: ((e.user, e.product), 1))

counts = pairs.reduceByKey(lambda a,b: a+b)
```

The resulting interaction strength can be passed to a ranking component.

4.4 Recommendation Agent

The recommendation agent observes current user behaviour, consults candidate products and selects a ranked list. A session agent can react immediately to a newly viewed product. A learning agent can use later clicks or purchases as feedback to update preference estimates.

4.5 Dynamic Updating

Dynamic recommendation requires a feedback loop. If a user repeatedly ignores a category, its weight can be reduced. If the user purchases an item, related products may receive increased relevance. Real-time events are therefore not merely stored; they become signals that alter subsequent recommendations.

4.6 Scalability and Fault Tolerance

Partitioning by user ID or product ID can distribute work. Caching frequently used features can reduce repeated computation. RDD lineage provides recovery when partitions are lost. For very large recommendation workloads, Spark's higher-level DataFrame and machine-learning APIs may be preferable, while RDDs remain useful for custom transformations.

4.7 Reverse-Engineered Architecture

Layer	Inferred Component	Purpose
Input	Clicks, searches, purchases	Capture user intent
Processing	PySpark RDDs	Clean and aggregate events
Features	User/product interaction features	Represent preferences
AI	Ranking/recommendation model	Score candidates
Agent	Recommendation agent	Select and present items
Feedback	New interactions	Continuously improve recommendations

5. QUESTION 2 – REVERSE ENGINEERING A SMART TRAFFIC MANAGEMENT SYSTEM

Scenario: A city-wide traffic system adapts signals in real time according to vehicle density and congestion patterns.

5.1 Data Collection

- Roadside cameras and computer-vision counters.
- Inductive loops or other vehicle sensors.
- GPS and connected-vehicle telemetry where available.
- Traffic signal controller status.
- Historical traffic flow and congestion records.
- Emergency vehicle and incident information.

5.2 Reconstructed RDD Streaming Pipeline

Sensor Events → Time Window → RDD → Cleaning → Vehicle Count → Lane/Road Aggregation → Congestion Score → Traffic Agent → Signal Action

Each short interval of events can be represented as a distributed batch. The system may aggregate vehicle counts by intersection and direction. `reduceByKey()` can calculate total vehicles per road segment, while `filter()` can discard corrupt sensor records.

5.3 Congestion Feature Construction

Feature	Interpretation
Vehicle count	Number of vehicles observed
Average speed	Lower speed indicates congestion
Queue length	Approximate vehicles waiting at a signal
Waiting time	Measures delay
Lane occupancy	Fraction of road space occupied
Historical pattern	Expected flow for time/day

5.4 Intelligent Agent Interaction

An intersection agent receives local traffic measurements and decides whether the current signal cycle should be maintained, extended or shortened. Neighbouring intersection agents can exchange congestion information. A corridor-level coordinator can prevent one intersection from improving its own queue while creating a severe queue downstream.

5.5 Decision Under Time Constraints

Traffic decisions must be produced rapidly. Therefore, the critical path should use lightweight aggregation and precomputed thresholds or models. Historical model training can occur offline, while current traffic features are processed online. A safety policy should always override an AI action if a sensor failure or unsafe condition is detected.

5.6 Fault Handling

If a sensor stops reporting, the system can mark it unavailable and estimate traffic from neighbouring sensors or historical values. Distributed processing prevents one failed worker from necessarily stopping the entire pipeline. Agent policies should include safe fallback signal timings.

5.7 Reconstructed Architecture

Stage	Operation
-------	-----------

Sense	Capture vehicle and road conditions
Distribute	Send events to processing nodes
Transform	filter/map/aggregate RDD records
Infer	Calculate congestion and predicted flow
Decide	Intersection/corridor agent selects signal action
Act	Controller changes signal timing
Feedback	Observe resulting traffic state

6. QUESTION 3 – REVERSE ENGINEERING A FRAUD DETECTION PIPELINE

Scenario: A banking system flags potentially fraudulent transactions within milliseconds. The system must combine distributed data processing, feature extraction, anomaly detection and rapid alerting.

6.1 Transaction Inputs

- Transaction amount and currency.
- Account and merchant identifiers.
- Timestamp and transaction frequency.
- Device and channel information.
- Approximate geographic or region information.
- Historical spending behaviour.
- Authentication and risk signals.

6.2 Distributed Pipeline

Transaction Event → Validation → Feature Extraction → Historical Feature Lookup → Risk Scoring → Fraud Agent → Alert/Block/Review → Feedback

6.3 RDD Operations

A transaction RDD can be filtered for valid schema and mapped into risk features. `reduceByKey()` can compute recent transaction counts or total spend per account. `join()` can combine the current transaction with historical account features. The output can then be passed to an anomaly detector or classification model.

Example logic:

```
transactions.filter(valid_transaction)

    .map(to_features)

    .map(lambda x: (x.account_id, x.amount))

    .reduceByKey(lambda a,b: a+b)
```

The aggregate is one input to a larger risk-scoring process.

6.4 Feature Extraction

Feature Group	Example Signal
Amount	Deviation from normal amount
Velocity	Number/value of transactions in a short interval
Merchant	Unusual merchant category
Device	New or previously unseen device
Location	Unexpected region or travel pattern
History	Distance from account baseline

6.5 Anomaly Detection

The system may combine supervised classification with anomaly detection. A classifier can estimate the probability of fraud using known labelled transactions. An anomaly model can identify behaviour that is statistically unusual even when no labelled example exists. A rules engine can add deterministic controls for high-risk conditions.

6.6 Fraud Alert Agent

A fraud agent receives the risk score and context, applies policy thresholds and chooses an action. Low-risk transactions may be approved. Medium-risk transactions may trigger additional authentication. High-risk transactions may be held or blocked for investigation. The agent should log its decision and reason to support auditing.

6.7 Latency and Reliability

Because fraud decisions occur close to transaction time, only compact features and efficient models should be placed on the critical path. Frequently used account statistics can be cached. RDD lineage and distributed workers improve resilience, while a separate durable event log should preserve the original transaction events.

7. QUESTION 4 – REVERSE ENGINEERING A PERSONALIZED LEARNING SYSTEM

Scenario: An online learning platform adapts content difficulty according to student performance and engagement.

7.1 Learner Data

- Quiz and assignment scores.
- Question-level correctness.
- Time spent on lessons.
- Attempts and hints used.
- Video completion and navigation behaviour.
- Topic history and prerequisite mastery.
- Student feedback and learning goals.

7.2 Reconstructed Processing Pipeline

Learner Events → RDD → Cleaning → Student-Topic Aggregation → Mastery Features → Difficulty Estimation → Learning Agent → Content Selection → Student Feedback

7.3 RDD Transformations

A map transformation can convert raw events into (student, topic, score) records. filter() can remove incomplete events. reduceByKey() can aggregate scores and attempts. join() can combine student performance with topic metadata. These transformations produce features such as average accuracy, recent accuracy and engagement rate.

7.4 Example Learning Features

Feature	Use
Recent accuracy	Detect current mastery
Long-term accuracy	Measure stable knowledge
Attempts	Identify difficulty or persistence
Response time	Indicate fluency or struggle
Hint usage	Identify support needs
Topic progression	Choose prerequisite or advanced content

7.5 Intelligent Learning Agent

The learning agent maintains a learner state and selects the next activity. If performance is high, it may increase difficulty or move to a related concept. If performance falls, it can provide revision material, hints or simpler questions. A feedback agent observes whether the chosen intervention improved later performance.

7.6 Adaptive Feedback Loop

The central loop is Observe → Assess → Select → Deliver → Measure → Update. This is an agent-based control loop. The agent should avoid making decisions from one poor result alone; recent evidence can be combined with historical performance to produce a more stable learner model.

7.7 Fairness and Privacy

Educational systems process sensitive behavioural data. Only necessary information should be collected, access should be controlled and student data should be protected. The adaptive model should be monitored for systematic differences in recommendations or difficulty that cannot be justified by learning evidence.

8. QUESTION 5 – REVERSE ENGINEERING A CLOUD-BASED AI MONITORING SYSTEM

Scenario: A cloud platform automatically scales resources and detects system anomalies in real time.

8.1 Data Sources

- Application logs.
- CPU, memory and storage metrics.
- Network throughput and latency.
- Request rates and error rates.
- Container or virtual-machine events.
- Deployment and configuration changes.
- Security and access logs.

8.2 Reconstructed Architecture

Cloud Resources → Metrics/Logs → Distributed Collection → PySpark RDD Processing → Feature Aggregation → Anomaly Detection → Monitoring Agent → Scaling/Remediation Agent → Cloud Infrastructure → Feedback

8.3 Large-Scale Log Processing

Logs can be parsed using `map()` into structured records. `filter()` can remove irrelevant entries. `reduceByKey()` can count errors by service or time window. `groupByKey()` can be used when grouped records are required, although aggregation operations are often more efficient. `sortBy()` can rank services by error frequency.

8.4 RDD Fault Tolerance

RDD lineage records the sequence of transformations used to produce a partition. If a worker fails, Spark can recompute lost partitions. This is important for monitoring because log processing should continue despite individual worker failures. Persistent storage should still be used for audit logs and long-term retention.

8.5 Anomaly Detection

Anomaly detection can operate on metrics such as CPU usage, latency, request rate and error count. A simple system can use thresholds, while an AI-based system can learn normal patterns and flag deviations. Multiple signals can be combined to reduce false alarms.

8.6 Intelligent Monitoring Agents

Agent	Role
Collector Agent	Observes logs and metrics
Anomaly Agent	Identifies abnormal patterns
Diagnosis Agent	Investigates likely causes
Scaling Agent	Adds/removes resources
Remediation Agent	Restarts or reconfigures services
Coordinator Agent	Prioritizes and coordinates actions

8.7 Autonomous Scaling Decision

A scaling agent may observe sustained high CPU, rising request latency and increasing queue depth. Instead of reacting to one metric, it can combine several features and estimate whether additional resources are necessary. After scaling, the monitoring agent observes whether latency and utilization return to acceptable levels. This closes the feedback loop.

8.8 Safety Controls

Autonomous cloud actions must be constrained by policies. The agent should have maximum scaling limits, cooldown periods, approval requirements for risky operations, rollback mechanisms and complete action logs. These controls prevent unstable feedback loops such as repeated scaling up and down.

9. COMPARATIVE ANALYSIS OF THE FIVE SYSTEMS

System	Main Data	RDD Role	Main Agent Action	Critical Constraint
Recommendation	Clicks, searches, purchases	User-product aggregation	Rank products	Personalization + scale
Traffic	Sensors, GPS, signals	Windowed traffic aggregation	Change signal timing	Very low latency
Fraud	Transactions, history	Feature aggregation	Approve/hold/block	Milliseconds + security
Learning	Scores, engagement	Student-topic features	Select content	Fair adaptation
Cloud Monitoring	Logs, metrics	Log/metric aggregation	Scale/remediate	Reliability + stability

9.1 Common Architecture

All five systems follow a common distributed-AI pattern: data acquisition, distributed processing, feature generation, inference, agent decision, action and feedback. Their differences arise mainly from data velocity, decision latency, consequences of errors and the type of action performed.

9.2 RDD Operations Across Applications

Operation	Recommendation	Traffic	Fraud	Learning	Cloud
map	✓	✓	✓	✓	✓
filter	✓	✓	✓	✓	✓
reduceByKey	✓	✓	✓	✓	✓
join	✓	✓	✓	✓	✓
sortBy	✓	✓	✓	✓	✓

10. CHALLENGES, SECURITY AND ETHICAL CONSIDERATIONS

10.1 Scalability

Distributed AI systems must handle increasing data volume without unacceptable increases in latency. Partitioning, parallel processing, caching and efficient aggregation are central techniques.

10.2 Data Quality

Missing, duplicated, delayed or corrupted events can produce incorrect decisions. Every pipeline should include validation, schema checks and monitoring of data quality.

10.3 Security

- Encrypt data in transit and at rest.
- Apply least-privilege access controls.
- Protect model and feature stores.
- Audit agent decisions and administrative actions.
- Detect tampering or abnormal access patterns.

10.4 Model and Agent Risks

- False positives and false negatives.
- Model drift as real-world behaviour changes.
- Feedback loops that amplify undesirable behaviour.
- Over-automation without human override.
- Biased recommendations or decisions.
- Insufficient explainability for high-impact actions.

10.5 Human-in-the-Loop Design

Agents should not be treated as infallible. High-impact actions can be routed to human reviewers. A robust architecture therefore combines autonomous routine actions with escalation policies for uncertain or high-risk situations.

11. CONCLUSION

The five reverse-engineering exercises demonstrate how modern AI applications can be reconstructed as distributed data-processing and intelligent-agent systems. Although the internal implementations of real commercial platforms are proprietary, observable behaviour allows a reasonable architecture to be inferred.

PySpark and RDD operations provide a conceptual foundation for processing large datasets in parallel. Transformations such as `map()`, `filter()`, `reduceByKey()` and `join()` convert raw events into useful features, while actions produce outputs for downstream analytics. RDD lineage contributes fault tolerance by allowing lost partitions to be recomputed.

Intelligent agents provide the decision-making layer. Recommendation agents rank products, traffic agents control signals, fraud agents issue risk responses, learning agents adapt educational content, and cloud agents monitor and scale infrastructure. In every case, the most important architectural feature is the feedback loop: the environment produces data, the distributed system analyses it, an agent acts, and the resulting behaviour becomes new data.

A well-designed distributed AI system therefore requires more than an accurate model. It must also provide scalable processing, low-latency decisions where required, fault tolerance, secure data handling, explainability, monitoring and safe agent policies. These principles support the design of reliable AI applications across commerce, transportation, finance, education and cloud computing.

12. REFERENCES

10. Apache Spark Documentation – Resilient Distributed Datasets (RDDs), transformations, actions and fault tolerance.
11. Apache Spark Documentation – PySpark programming guide and distributed data processing concepts.
12. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson.
13. IBM / industry literature on recommendation systems and large-scale personalization.
14. Research literature on intelligent transportation systems and adaptive traffic signal control.
15. Research literature on machine learning for financial fraud detection and anomaly detection.
16. Research literature on adaptive learning systems, learner modelling and intelligent tutoring.
17. Cloud computing literature on monitoring, autoscaling, anomaly detection and autonomous operations.

APPENDIX A – GENERIC PYSPARK RDD TEMPLATE

The following template illustrates how a reverse-engineered pipeline can be expressed in PySpark. It is intentionally generic so that the same pattern can be adapted to recommendations, traffic, fraud, learning or cloud monitoring.

```
from pyspark import SparkContext
```

```
sc = SparkContext.getOrCreate()
```

```
raw = sc.textFile("input.txt")
```

```
# 1. Parse records
```

```
records = raw.map(lambda line: line.split(","))
```

```
# 2. Remove invalid records
```

```
valid = records.filter(lambda r: len(r) >= 3)
```

```
# 3. Create key-value features
```

```
features = valid.map(lambda r: (r[0], float(r[2])))
```

```
# 4. Aggregate distributed data
```

```
aggregated = features.reduceByKey(lambda a, b: a + b)
```

```
# 5. Produce a small result
```

```
result = aggregated.take(10)
```

```
for item in result:
```

```
    print(item)
```

APPENDIX B – REVERSE ENGINEERING CHECKLIST

- Identify inputs and outputs.
- Identify data sources and event frequency.
- Identify distributed processing stages.
- Map each stage to suitable RDD transformations.
- Identify features and scoring logic.
- Identify intelligent agents.
- Describe agent goals, observations and actions.
- Explain feedback and continuous adaptation.
- Check latency, scalability and fault tolerance.
- Evaluate security, fairness, privacy and human oversight.